Model development, tuning and selection

A plan for building the training section the way a data-science team in a company would

Written 2026-10-11. Reference document; nothing in Sections 4–6 has been built yet.

1 Why this document

Two questions came up while preparing for interviews, and one decision followed from them.

1. **How can algorithms that have different hyperparameters be compared** (a decision tree against LightGBM, for example), and how is one of them chosen to forecast a new day?
2. **How should hyperparameter tuning be shown** so that it is a strong, defensible part of the project and not just a grid search that anyone can run?
3. **Decision:** build this as a proper section of the project, with the old fixed-settings selection replaced once the new one is shown to work.

2 How different algorithms are compared

2.1 The principle

The hyperparameters are never compared. Each algorithm is a different function with its own knobs (depth and leaf size for a tree; learning rate, number of leaves and sampling for LightGBM). What is compared is **how wrong the forecasts were on days the model had not seen**. A "model" in this comparison is the algorithm together with its tuning procedure.

2.2 The procedure

1. **Tune each algorithm on its own**, using only its training window. An inner validation block (the latest days of the training window, separated by a gap) picks the settings. All algorithms get the **same search effort**.
2. **Score all tuned algorithms the same way**: rolling-forward folds, same delivery days, same data, same metrics (sum of squared errors, MAE and RMSE in price units). Never a random split, which lets the future leak into the past.
3. **Test whether a difference is real.** The lowest MAE can be luck. The Diebold–Mariano test on daily errors says whether one model is better or whether the two cannot be told apart.
4. **Use a written selection rule.** If the leading models are not significantly different, prefer an average of them or the simpler model.
5. **Score the chosen model once on a locked test period** that played no part in tuning or selection. That is the honest accuracy to quote.
6. **For a new day, use the selected algorithm refitted on the latest data**, and repeat the selection on a schedule, because the best algorithm changes with the market.

2.3 "Which algorithm is best for one particular day?"

One day cannot answer this: a single day is mostly luck. What can be shown is how often each model wins across many days, hours and market regimes, and which model has been best recently. The error-weighted combination in the framework already uses the second idea (weights from the last 28 days of errors, lagged by two days).

2.4 Weakness to keep in mind

Equal *number* of trials is not equal *coverage*: LightGBM has eight settings to search and a decision tree two, so twelve draws cover the large space more thinly. The plan raises the budget and uses a smarter search for this reason.

3 Where the project stands

3.1 What the old pipeline does

The existing price forecasters (day-ahead, IDA1–3, XBID, aFRR, mFRR) share `ml_train_val_test_common.py`. Each algorithm (XGBoost, LightGBM, random forest) is fitted with **fixed, hand-set** parameters, scored on walk-forward folds, and the lowest mean MAE is chosen. A Wilcoxon test on per-fold errors is available.

3.2 What was added in Phases 6, 7 and 9

- **Phase 6:** one evaluation contract, rolling-origin backtest, Diebold–Mariano test with HAC variance, locked test period, leakage check. **Phase 7:** point-in-time feature catalogue and store.
- **Phase 9:** nested, time-aware tuning (12 random draws, defaults always included, re-tuned every 180 days), forecast combinations, and a comparison of window length and refit frequency.

3.3 Phase 9 results on the day-ahead price (development window, 1 015 days)

Model	MAE untuned	MAE tuned	Tuned vs untuned	DM p
Decision tree	20.28	19.82	better	0.012
Random forest	18.11	18.06	no clear difference	0.46
Gradient boosting	17.96	17.77	no clear difference	0.17
XGBoost	18.24	17.72	better	0.0003
LightGBM	18.29	**17.43**	better	< 0.0001

- **Tuning changes the ranking.** Untuned, gradient boosting led; tuned, LightGBM leads. This is the reason a comparison of fixed-setting models is not trustworthy.
- **Combinations** (mean, median, trimmed mean, inverse error, top three) are within 0.1 MAE of the best single tuned model and cannot be told apart from it (p from 0.22 to 0.99). They are insurance, not an improvement.
- **A longer calibration window helped as much as tuning.** LightGBM at defaults: 730 days 18.29, expanding window 17.56 (DM $p < 0.001$). Refit every 7, 30 or 90 days made no detectable difference. Whether tuning and the longer window add up is **not tested**.
- The tuned run took 7.7 hours (three jobs sharing the machine) against 21 minutes untuned.

Limits. One target and the development window only; the locked test period is untouched. Twelve random draws is a light search. The significance tests are not adjusted for the number of comparisons. A repeat run is recovering the tuning log that the first run failed to save.

4 Plan

The code goes in `common_layer/model_development/`; a showcase folder `model_development/` holds the README that tells the story, the tuning logs, the figures and one script that reproduces everything. Tests in `tests/test_model_development.py`. Tree models only (decision tree, random forest, gradient boosting, XGBoost, LightGBM); metrics SSE, MAE and RMSE, as already decided.

#	Stage	What is built	Done when
1	Experiment tracking	A local run registry (SQLite): every run stores model, feature set, window, settings, scores, data digest and code version.	Every number in the documents traces to a run.
2	Fair, stronger tuning	Same trial budget for every algorithm (for example 40). Smarter search (Optuna). Several rolling inner validation blocks instead of one 90-day block. Parallel trials.	Tuned never worse than default on validation; no leakage (existing tests plus new ones).
3	Tuning evidence	Log every trial. Figures: error against each setting, which settings matter, stability of choices across folds, default against tuned, overfitting gap (inner against outer error), time cost.	The tuning folder reads without running anything.
4	Algorithm comparison	Five tuned algorithms on the same days. Pairwise Diebold–Mariano matrix with Holm correction. Win rate by hour, month and regime.	The ranking states which differences are real.
5	Selection policy	A written rule in the config: lowest tuned MAE on the development window; if the top models are not significantly different, prefer an average or the simpler model.	The rule is applied by code, not by hand.
6	Locked test	The champion is scored once on the locked period; the digest is locked.	One honest number per target.
7	Champion registry	The fitted model, settings, feature-set version and digests are stored. A challenger replaces the champion only if it beats it significantly over the last N days.	Live code loads the champion from the registry.
8	Monitoring and retraining	Rolling error tracking, a drift flag, re-tuning every 180 days, refit every 30 days.	A report shows when the champion degrades.
9	Roll-out	Repeat for DA, IDA1, IDA2, IDA3, XBID, aFRR and mFRR.	One table of champions with locked-test scores.
10	Replace the old selection	Switch the dashboard and the phase forecasters from fixed-setting, lowest-MAE selection to the registry.	Existing pipeline tests still pass.

4.1 Order and cost

Stages 1–6 on the day-ahead price first, then 7–8, then 9, and 10 last, because stage 10 changes the live pipeline. Compute is the main cost: the first tuned run took 7.7 hours for one target at 12 trials. A larger budget needs parallel trials and re-tuning only every 180 days; the time will be measured before it is promised.

5 What happens to the old code

Nothing is deleted before the new section has been shown to work. The old code runs the live pipeline, and it is the baseline that the new framework has to beat.

- **Replaced at stage 10:** the fixed-settings fitting functions and lowest-MAE selection in `ml_train_val_test_common.py`; the per-forecaster `*_train_val_test.py` scripts; and `tests/test_model_selection_metrics.py`, which is rewritten for the new selection.
- **Kept:** the forecaster modules (they load a model and predict, but from the registry); the PV and reservoir-inflow forecasters (not price targets, outside this programme unless requested); the walk-forward utilities until each use has been checked.
- **Safe procedure:** build alongside the old code; compare old against new on the same days and record it; switch one forecaster at a time, running its tests after each; delete the old code in a separate commit so it can be restored from history.

6 Decisions needed

1. **Optuna** as a dependency, or stay with random search? (Recommended: Optuna.)
2. **Registry:** a local SQLite registry, or MLflow? (Recommended: local first, with a clean path to MLflow.)
3. **First target:** day-ahead price, or another? (Recommended: day-ahead.)
4. **Stage 10:** agreement to change the existing phase forecasters at the end.

7 Short answers for an interview

"How do you compare algorithms with different hyperparameters?" I do not compare the hyperparameters. I tune each algorithm on its own with the same search effort, using only training data and a time-ordered validation block, and then I compare the tuned models by out-of-sample error on the same days. I test the difference with Diebold–Mariano, and if two models cannot be separated I average them or take the simpler one.

"How do you pick the model for a new day?" The selected algorithm, refitted on the latest data, with a locked test period that was never used for selection, and a scheduled re-check, because the best algorithm changes with the market.

"Did tuning help?" For LightGBM and XGBoost, yes (4.7% and 2.8% lower MAE, significant). For the forest and gradient boosting, not significantly. A longer training window helped as much as tuning, and an average of models was as good as the best single model.